

Reinforcement Learning for Real-Time Vision-Language-Action Policies

Perry Dong^{*,†} Kuo-Han Hung^{*} Dorsa Sadigh Chelsea Finn

Stanford University

<https://pd-perry.github.io/real-time-expo-ft/>

Real-Time EXPO-FT: RL for Real-Time VLA Policies

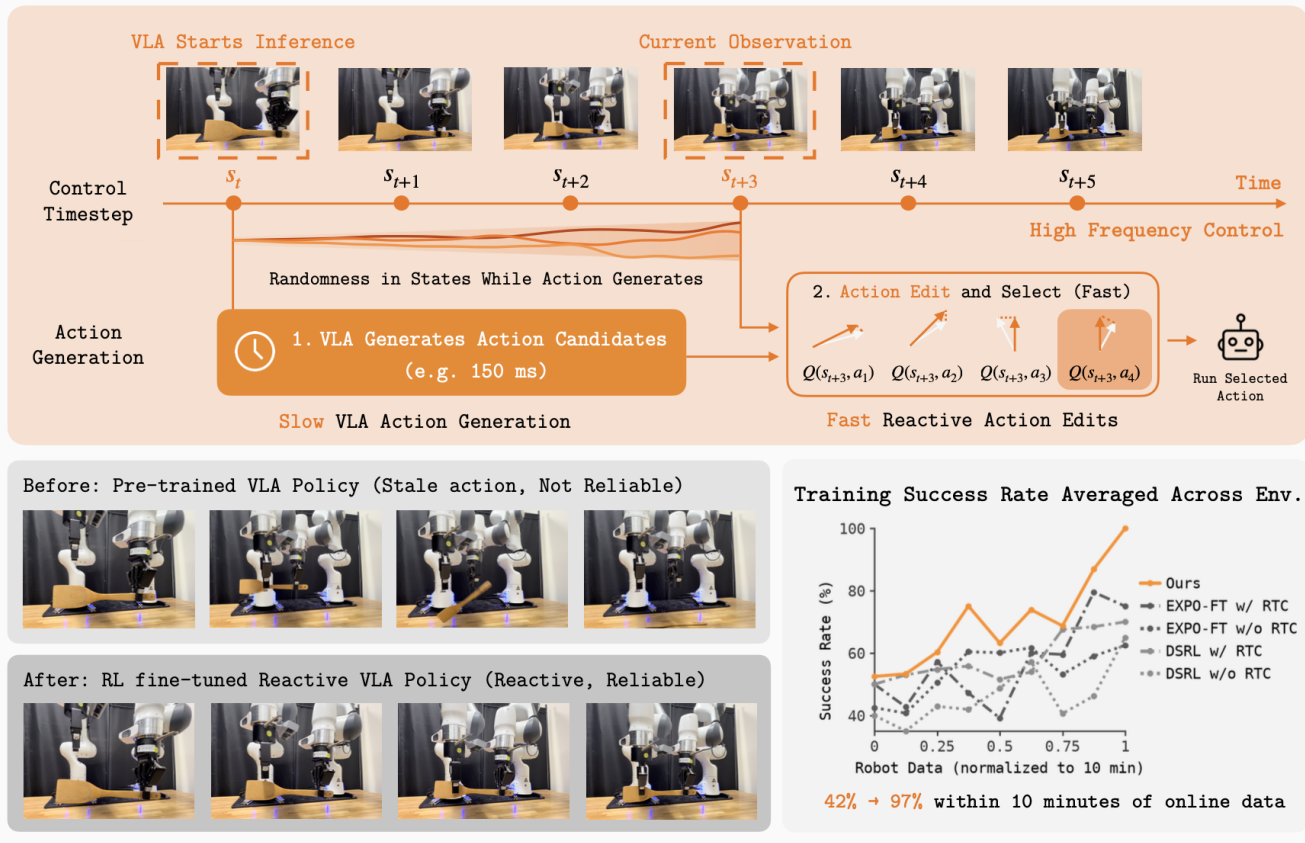


Fig. 1: **Overview of Real-Time EXPO-FT.** Real-Time EXPO-FT addresses the latency of large VLA models by combining slow action generation with fast reactive control. **Bottom left:** Task success rates before and after applying Real-Time EXPO-FT. **Bottom right:** Training success rates on real-world tasks compared with baselines.

Abstract—Reinforcement learning fine-tuning on top of large, pretrained Vision-Language-Action (VLA) models offers promise for highly reliable robot deployment. However, because of their scale, modern VLA models suffer from high inference latency, so the observation used to select an action is often stale by execution time, creating a distribution shift that can substantially degrade reliability and performance. Prior work has explored asynchronous policy execution to reduce the effect of latency, but these methods are mostly built on imitation learning and offer no mechanism for moving beyond the training distribution toward higher reliability. We close

this gap by enabling RL fine-tuning that meets the real-time control requirements of dynamic real-world manipulation. Our approach builds on EXPO-FT, a framework for sample-efficient, reliable VLA fine-tuning with reinforcement learning, and decouples slow, expressive action generation from fast, reactive action edits: a large pretrained VLA proposes action chunks using its strong behavior prior, while a lightweight edit policy performs fast, reactive decision-making by editing actions in response to changes in state, conditioned on the latest observation. We instantiate this as Real-Time EXPO-FT, an RL framework for finetuning real-time VLA policies. On the Kinetix benchmark, Real-Time EXPO-FT enables a delayed policy to achieve the best performance among delayed and non-delayed methods in 10 out of 10 environments. On four

^{*}Equal contributions.

[†]Corresponding author: perrydong@stanford.edu

dynamic real-world tasks—robot object passing, ball balancing, table soccer kicking, and dynamic object picking—with online robot data capped at 10 minutes, Real-Time EXPO-FT improves average policy performance from 42% to 97%, all without human intervention, demonstrating rapid, sample-efficient adaptation to challenging real-world dynamics.

I. INTRODUCTION

Reinforcement learning fine-tuning on top of large, pre-trained vision-language-action (VLA) models offers promise toward highly reliable robot deployment in the real world [19, 35]. However, the scale behind pretrained VLA models that makes it a strong behavior prior cuts both ways. While the large capacity enables complex, multi-step behaviors across diverse tasks and embodiments, it also inflates inference latency, and the physical world does not pause while the robot computes its next action. As a result, for the most capable models, the observation used to infer an action is often not the observation at the time of execution, creating a distribution shift that can substantially degrade reliability and performance. In this work, we study RL fine-tuning of VLA policies for real-time execution, trained explicitly considering inference latency of a large VLA model so that the policy can retain maximal reactivity to changes in the environment while achieving higher reliability with reinforcement learning.

Prior work on asynchronous policy execution has explored alternative inference schemes [15, 49], alternative training schemes [16], or both [29, 47] to enable reactive control under inference latency. A representative example is real-time chunking (RTC) [15, 16], which uses action inpainting to generate the next action chunk while the current one is still executing. While such methods enable smooth inference despite latency, they offer no natural mechanism for moving beyond the training data distribution. While methods like RTC can be used for reinforcement learning as a base policy, directly applying RL leaves performance on the table since actions are predicted from previous observations, and thus cannot fully capture the reliability gains that reinforcement learning affords by improving on the base policy. This gap motivates our work of enabling reinforcement learning fine-tuning for real-time policies, so as to simultaneously obtain the reliability benefits of reinforcement learning and the reactivity benefits of real-time execution.

Our key insight is that reinforcement learning can allow the VLA policy to be maximally reactive by editing the action to be executed using the latest observation at the time of execution, with the base action being generated from the VLA using previous observations given the inference latency. We build on EXPO-FT [35], a system for sample-efficient, reliable VLA fine-tuning with reinforcement learning that features a large VLA base policy and a small edit policy. Specifically, we turn pretrained VLA policies into reactive controllers by decoupling slow action generation from fast, observation-conditioned action generation. We assign the base VLA, a large policy with a strong behavior prior, to be responsible for initial action chunk generation, which can incur substantial inference delay, and the edit policy

to perform fast, reactive decision-making by editing actions in response to changes in the observation. Concretely, at inference, the VLA proposes candidates of action chunks; once the actions have been generated and a delay has elapsed, the edit policy transforms the remaining actions taking into account the latest observation and selects the chunk of remaining actions with the highest Q-value. We instantiate these ideas as Real-Time EXPO-FT, a reinforcement learning (RL) framework for fine-tuning real-time VLA policies.

Our key contribution is a framework for reinforcement learning fine-tuning of real-time policies. We evaluate our approach in both simulation and the real world on highly dynamic and stochastic tasks that require both fast reaction and accurate control. In simulation, we evaluate on the Kinetix benchmark [26] following prior work on real-time control, and empirically show that Real-Time EXPO-FT enables a delayed policy to achieve the best performance among delayed and non-delayed methods in 10 out of 10 environments. In the real world, we evaluate on four dynamic manipulation tasks, including robot object passing, ball balancing, table soccer kicking, and dynamic object picking. These tasks involve rapid state changes, stochastic outcomes, and tight timing requirements. Across these tasks, capping the online robot data to 10 minutes, our method improves the average evaluation policy performance from 42% (12.5/30) to 97% (29/30), all without human intervention during training, demonstrating rapid and sample-efficient adaptation to challenging real-world dynamics.

II. RELATED WORK

Deep Reinforcement Learning for Robotic Manipulation. Reinforcement learning has been widely used to improve manipulation policies through direct interaction with the environment [1, 2, 4–6, 10, 14, 23, 43]. However, real-world interaction is costly, making sample efficiency a fundamental challenge. Prior work addresses this challenge through algorithmic design [3, 7, 9, 12, 18, 24] to enable effective policy improvement from limited experience. These methods typically optimize lightweight Gaussian policies, which provide low inference latency and support high-frequency control for dynamic manipulation tasks. However, their limited policy capacity prevents them from leveraging the broad behavioral priors of large pretrained models, often requiring substantial number of samples and small breadth of initial states. Our work bridges this gap by enabling sample-efficient RL adaptation of pretrained VLAs while satisfying the latency and reactivity constraints of real-time manipulation.

Reinforcement Learning for Vision-Language-Action Models. Recent work has explored reinforcement learning for finetuning pretrained vision-language-action models. However, modern VLAs [19, 20, 27, 30, 33] often employ expressive generative policies, such as diffusion or flow-matching policies, making conventional continuous-control RL algorithms difficult to apply directly. Several works address this mismatch by developing RL algorithms tailored to diffusion or flow-based policies [11, 13, 28, 36–38, 40, 41, 44, 48, 52]. Multiple

online RL methods use on-policy algorithms to finetune the VLA [28, 34, 52], which can require extensive environment interaction. Consequently, recent work has increasingly explored off-policy RL for VLA finetuning [17, 22, 31, 35, 39, 50, 51], as it can reuse previously collected experience and substantially improve sample efficiency. Our work builds on EXPO-FT [35], a system for sample-efficient, reliable finetuning of VLA models with reinforcement learning, and we focus on a largely unexplored challenge: enabling RL finetuning while meeting the real-time control requirements of dynamic real-world manipulation. In particular, whereas prior work have shown improving policy performance and sample efficiency from RL fine-tuning, we study how to obtain these benefits while achieving sufficiently low inference latency for high frequency control in dynamic environments.

Real-Time Inference for Vision-Language-Action Models.

VLA models typically incur substantial inference latency due to their large size, limiting their ability to support high-frequency control. Prior work has explored system-level techniques to accelerate VLA inference [25, 32, 45, 46], including model compression [32], kernel-level optimization [25], accelerated sampling [45], and speculative inference [46]. Another line of work modifies the inference or training procedure to enable more responsive execution. One strong, widely used approach is real-time chunking (RTC) [15], which uses action inpainting to generate the next action chunk while the current chunk is still being executed, thereby enabling asynchronous VLA execution. Subsequent works incorporate this capability directly into policy training. Training-Time RTC [16] and VLASH [49] modify the standard VLA training procedure by incorporating action conditioning or future-state prediction during training to enable asynchronous action generation during inference. $\pi\mathbf{R}^2$ [47] combines a fast proprioceptive channel alongside a slow updated vision-language channel. Other approaches use auxiliary policies and correction modules to refine actions between successive VLA updates [29, 42]. Different to these approaches, we focus on the reinforcement learning setting with the goal of enabling reinforcement learning fine-tuning for real-time policies. These prior methods are general and can in principle be combined with RL post-training; our insight, however, is that the structure of modern RL algorithms enables us to design an approach that is even more performant (Figure 3 and Figure 5).

III. BACKGROUND

We consider the standard reinforcement learning framework, where problems are modeled by a Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, r, T, \gamma, \rho)$. In the MDP, $\mathcal{S}$ is the state space and $\mathcal{A}$ is the action space. At each timestep t , the agent selects action $a_t \in \mathcal{A}$ according to policy $\pi(\cdot | s_{t-d}, s_t)$ and receives scalar reward $r(s_t, a_t) \in \mathbb{R}$. The environment transitions following the transition dynamics of the MDP $s_{t+1} \sim T(\cdot | s_t, a_t)$, with starting state initialized from $s_0 \sim \rho(\cdot)$. The tuple (s_t, a_t, r_t, s_{t+1}) is added to the replay buffer $\mathcal{D}$ for learning. The goal of reinforcement learning is to maximize the expected discounted return

$\mathbb{E}_\pi \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right]$, where $\gamma \in [0, 1]$ is the discount factor. We consider the problem of reinforcement learning fine-tuning of VLA models under a real-time constraint, where policy inference time needs to be faster than the control frequency f . Because VLAs are large models, inference itself is costly. We denote the delay of timesteps from inference as d . Modern VLAs often employs action chunking, predicting a sequence of H future actions $a_{t:t+H}$ at each timestep and executing $C \leq H$ at each timestep. We assume the delay d is less than or equal to execution length C .

Training-Time Real-Time Chunking [16]. When inference incurs a delay of d timesteps, the first d actions of a newly predicted chunk cannot be executed, since by the time they are produced the environment has already advanced to $t + d$. Training-time RTC [16] addresses this issue in the imitation learning setting by additionally executing actions $a_{t+C:t+C+d}$ from the previous chunk while inference for the new chunk is in progress, and conditioning the new prediction on this committed action prefix. Specifically, given the original state s_t and the committed action prefix $a_{t:t+d}^{\text{prev}}$, the policy predicts the remaining actions as $a_{t+d:t+d+H} \sim \pi(\cdot | s_t, a_{t:t+d}^{\text{prev}})$. By conditioning on both the current state and the already-committed actions, the policy can account for inference latency and produce a coherent continuation of the action chunk. During training, the delay d is randomly sampled across a range of values so that the policy learns to remain robust to varying inference latencies at deployment time.

EXPO and EXPO-FT [35, 38]. To finetune the VLA policy with RL, we build on EXPO [38], a recently proposed RL algorithm that is both highly sample-efficient and stable for training expressive policies. Classical sample-efficient RL algorithms are designed around Gaussian policies and cannot be directly applied to pretrained VLAs, which typically use flow or diffusion policies; EXPO instead provides a principled foundation for RL fine-tuning in this regime.

EXPO couples two parameterized policies. The first is a base flow policy—in our setting the VLA model π_{VLA} , obtained through supervised training—and the second is a small edit policy π_{edit} whose objective is to maximize the learned Q-value:

$$\mathcal{L}(\pi_{\text{edit}}) = -\mathbb{E}_{(s_t, a_t) \sim \mathcal{D}, \hat{a}_t \sim \pi_{\text{edit}}} \left[Q_\phi(s_t, a_t + \hat{a}_t) - \alpha \log \pi_{\text{edit}}(\hat{a}_t | s_t, a_t) \right] \quad (1)$$

Rather than altering the base action outright, π_{edit} predicts a bounded edit $\hat{a}$ restricted to $[-\beta, \beta]$, which is summed with the base action a to yield the edited action $\tilde{a} = a + \hat{a}$, transforming the base action to a higher value distribution. Confining the Q-function signal to this edit avoids backpropagation of the Q-value to the VLA backbone and also anchored to actions already close to optimal. During rollout or backup target construction, EXPO uses an on-the-fly (OTF) policy that chooses the highest-value action candidate from the base and edited actions:

$$\tilde{a}^* = \arg \max_{a \in \bigcup_{i=1}^N \{a_i, \tilde{a}_i\}} Q_\phi(s, a) \quad (2)$$

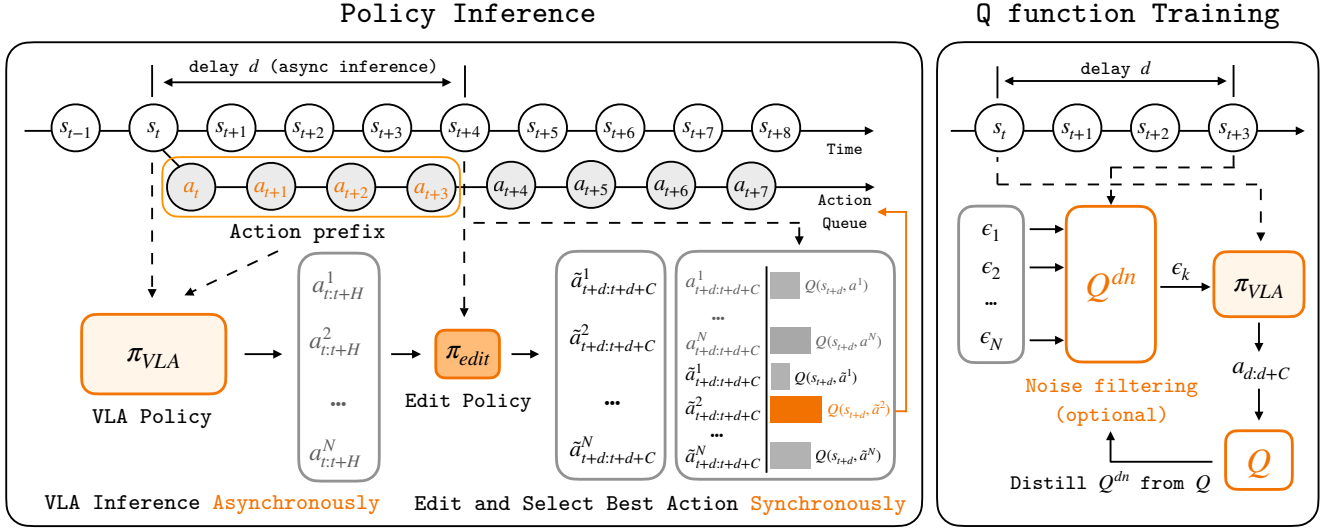


Fig. 2: **Left: Real-time policy inference of Real-Time EXPO-FT.** Real-Time EXPO-FT decouples slow VLA action generation from fast, reactive decision-making. While the robot executes the current action chunk, the VLA asynchronously generates multiple candidate action chunks. Once new actions are required, a lightweight edit policy refines candidates using the latest observation, and a learned Q-function selects the highest-value action chunk for execution. **Right: Noise-level filtering during Bellman backup.** During Bellman backup, we filter samples in the noise space to reduce training compute.

The critic itself is fit by temporal-difference:

$$\mathcal{L}(\phi) = \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left[(r_t + \gamma Q_{\phi'}(s_{t+1}, \tilde{a}_{t+1}^*) - Q_{\phi}(s_t, a_t))^2 \right] \quad (3)$$

EXPO-FT [35] is a system on top of EXPO to finetune VLA models, incorporating human-in-the-loop and action chunking. We build directly on top of EXPO-FT. While human intervention can provide useful corrective signals, we do not use human intervention for the experiments in this paper.

IV. REAL-TIME EXPO-FT

In this section, we present a complete framework for finetuning VLA models with reinforcement learning for real-time robotic control. Our goal is to efficiently enable pretrained VLA policies to reach high reliability in dynamic environments. We first formalize the learning setting and objective (Section IV-A), then introduce our real-time RL algorithm for VLA models (Section IV-B), and finally describe the training procedure (Section IV-C).

A. Problem Statement

We consider the problem of finetuning vision-language-action models π_{VLA} using reinforcement learning for real-time robotic control. Policy inference, especially for a large policy, may take non-negligible time, such that naively executing the predicted actions with a delay will result in distribution shift and the action executed at time $t + d$ has to be computed from earlier observations to mitigate the compute latency.

A common simplifying assumption in prior work is that policy inference is effectively instantaneous, so the action computed from an observation can simply be executed once

inference finishes. This assumption is increasingly untenable for large VLA policies, and it is especially costly to make during RL fine-tuning because standard RL assumes the Markov property, that the executed action is a function of the current state, $a_t \sim \pi(\cdot | s_t)$; under latency the action at t is actually a function of s_{t-d} , so the delayed process is no longer Markovian in s_t , and applying standard RL updates as if can bias credit assignment. This is specifically a problem when running RL for high reliability, where actions are refined precisely. Treating inference latency as negligible therefore risks significantly undermining the reliability that RL fine-tuning can deliver, motivating the need for an explicit solution.

For tasks that require real-time execution, we empirically observe that existing π_{VLA} models cannot achieve satisfactory success rates out of the box. Therefore, following the standard online RL finetuning practices, we assume access to a small offline dataset of expert demonstrations, $\mathcal{D}_0$, collected via either human teleoperation or scripted policies operating at the same control frequency. We adopt a sparse binary reward $r \in [0, 1]$ indicating successful task completion. The task completion classifier may be either rule-based or learned. Observations consist of multi-view RGB images from a wrist-mounted camera and a fixed side-view camera, augmented with the robot’s proprioceptive state. Policy parameters are updated either after every environment step, at the end of each episode, or at fixed episode-batch intervals. The objective is to maximize the task success rate.

To address the latency of π_{VLA} under high-frequency control, rather than optimizing the inference latency of the VLA model itself, we focus on asynchronously fine-tuning and executing the policy while maintaining a fixed real-

time control frequency. Suppose the VLA model requires t seconds for each inference and the robot operates at a control frequency of f Hz. The inference process therefore incurs a delay of approximately $d = \lfloor t \times f \rfloor + 1$ control steps, meaning that to execute a new action at timestep $t + d$, inference must be initiated approximately d control steps earlier, at timestep t . For typical hardware and VLA models, we assume $1 \leq d \leq C$, where C denotes the execution horizon of the policy. This inference delay creates a mismatch between the observation used to initiate VLA inference and the robot state at which the resulting action is eventually executed. Our goal is therefore to fine-tune the VLA policy using RL to explicitly account for this delay and improve policy performance under real-time control. In the following section, we discuss how these challenges are addressed in our approach, Real-Time EXPO-FT.

B. RL for Real-Time Vision-Language-Action Policies

We build on EXPO-FT [35] as a sample efficient RL fine-tuning framework. Real-Time EXPO-FT addresses the real-time inference challenge by decoupling action generation into two timescales: a slow, asynchronous step that generates candidate action chunks ahead of execution, and a fast, synchronous step that edits and selects among the highest value candidates at the time of execution, conditioned on the most recent observation. The method is illustrated in Figure 2. **Asynchronous VLA Action Generation.** The VLA is a large, expressive model for capturing behaviors, and because of its scale, calling the model during inference introduces a delay of d steps. We therefore begin inference from π_{VLA} asynchronously at time t when d steps remain in the currently queued action chunk. We sample multiple candidate action chunks from the observation s_t . Following the training-time RTC formulation [16], the actions from the previous chunk, $a_{t:t+d}^{\text{prev}}$, that are executed during the VLA inference window are inpainted as conditioning input when sampling future actions. Specifically, for each candidate i , we sample:

$$a_{t:t+H}^i = \pi_{\text{VLA}}(s_t, a_{t:t+d}^{\text{prev}}, \epsilon^i), \quad \epsilon^i \sim p(\epsilon) \quad (4)$$

where ϵ^i denotes the sampling noise used to produce diverse VLA candidates. Since the first d actions correspond to the inference-delay window, we retain only the subsequent C -step action segment $a_{t+d:t+d+C}^i$ for each action candidate.

Fast, Synchronous Edits. Once the environment arrives at the latest observation s_{t+d} for execution, a lightweight edit policy $\pi_{\theta}^{\text{edit}}$ transforms each candidate based on the latest observation to account for state changes during the inference delay and maintain reactivity:

$$\hat{a}_{t+d:t+d+C}^i \sim \pi_{\theta}^{\text{edit}}(\cdot \mid s_{t+d}, a_{t+d:t+d+C}^i) \quad (5)$$

The edited actions are $\tilde{a}_{t+d:t+d+C}^i = \hat{a}_{t+d:t+d+C}^i + a_{t+d:t+d+C}^i$. Finally, the action critic Q_{ϕ} evaluates both the original and edited candidates under the current state and

selects the highest-value action chunk for execution:

$$\tilde{a}_{t+d:t+d+C}^* = \arg \max_{a \in \bigcup_{i=1}^N \{a_{t+d:t+d+C}^i, \tilde{a}_{t+d:t+d+C}^i\}} Q_{\phi}(s_{t+d}, a) \quad (6)$$

This design enables expensive VLA inference to run asynchronously in the background while preserving fast, state-aware correction and selection immediately before execution.

Training Objective. We now describe how the action critic Q_{ϕ} , edit policy π_{edit} , and VLA π_{VLA} are updated. We use $Q_{\phi'}$ to denote the target critic.

We fine-tune π_{VLA} with the Training-Time RTC objective [16] on both offline demos and online data from rollouts. Following RTC, we simulate inference delay during training by splitting each ground-truth action chunk into a d -step action prefix and a remaining action postfix. The prefix is provided to the policy as clean, non-noisy actions with its flow-matching timesteps set to 1, while noise is added only to the postfix. The flow-matching loss is masked to the postfix:

$$\begin{aligned} A_t^{\tau} &= \tau A_t + (1 - \tau)\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \\ \mathcal{L}_{\text{BC}}(\pi_{\text{VLA}}) &= \mathbb{E}_{(s_t, A_t) \sim \mathcal{D}} \left[\left\| \mathbf{m}_d \odot (v_{\text{VLA}}(A_t^{\tau}, s_t, \tau_d) - (\epsilon - A_t)) \right\|_2^2 \right] \end{aligned} \quad (7)$$

where $\mathbf{m}_d$ masks out the first d actions from the loss and τ_d assigns flow-matching timestep 1 to the prefix and τ to the postfix. We use the specific execution delay d for the state after first chunk, and 0 for the first $[0, C]$ state in each episode during training.

We train the edit policy to bring the base actions from the VLA toward higher value regions like in EXPO [38] and EXPO-FT [35]:

$$\begin{aligned} \mathcal{L}(\pi_{\text{edit}}) &= -\mathbb{E}_{(s_t, a_{t:t+C}) \sim \mathcal{D}} \left[Q_{\phi}(s_t, a_{t:t+C} + \hat{a}_{t:t+C}) \right. \\ &\quad \left. - \alpha \log \pi_{\text{edit}}(\hat{a}_{t:t+C} \mid s_t, a_{t:t+C}) \right] \end{aligned} \quad (8)$$

Because the edit is conditioned on the latest observation, the combined base-plus-edit policy remains Markovian with delays. We train the critic to fit a chunk-level temporal-difference backup, in which one transition spans the full execution horizon C and the bootstrap term is evaluated at the next chunk $\tilde{a}_{t+C:t+2C}^*$,

$$\begin{aligned} \mathcal{L}(Q_{\phi}) &= \mathbb{E}_{(s_t, a_{t:t+C}, s_{t+C}) \sim \mathcal{D}} \left[\left(r_t + \gamma Q_{\phi'}(s_{t+C}, \tilde{a}_{t+C:t+2C}^*) \right. \right. \\ &\quad \left. \left. - Q_{\phi}(s_t, a_{t:t+C}) \right)^2 \right] \end{aligned} \quad (9)$$

Constructing the next chunk $\tilde{a}_{t+C:t+2C}^*$ is computationally expensive, as naively sampling N candidates from π_{VLA} requires N full VLA rollouts. We instead (optionally) search in noise space using a lightweight critic Q_{ψ}^{dn} , which scores candidate noise without denoising full actions, following FASTER [40]. At backup time, we select the best noise, decode it once with π_{VLA} to obtain a, generate K edits with π_{edit} , and use the target critic to select the final action chunk,

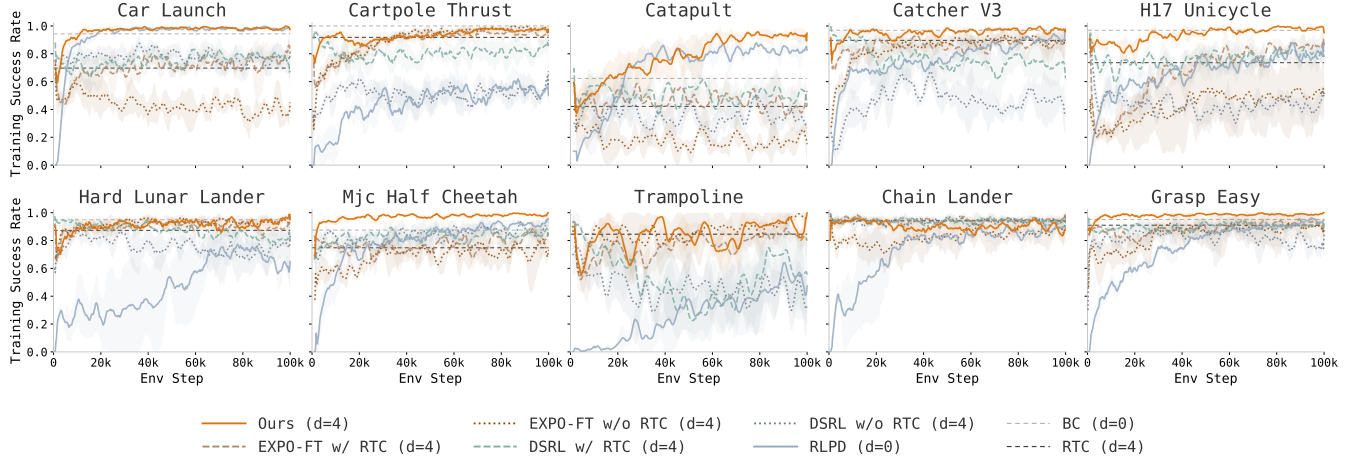


Fig. 3: **Training success across all 10 Kinetix [26] simulation tasks.** The vertical dashed lines indicate the evaluation performance of two BC baselines, one with no inference delay and one with a 4-step delay. Each setting is run with 4 seeds.

$$\begin{aligned}
 \epsilon^* &= \arg \max_{i \in [N]} Q_{\psi}^{\text{dn}}(s_{t+C-d}, \epsilon_i), \\
 a &= \left[\pi_{\text{VLA}}(s_{t+C-d}, a_{t+C-d:t+C}^{\text{prev}}, \epsilon^*) \right]_{d:d+C}, \\
 \tilde{a}_{t+C:t+2C}^* &= \arg \max_{a' \in \{a, a+\hat{a}\}} Q_{\phi'}(s_{t+C}, a')
 \end{aligned} \tag{10}$$

where $\epsilon_i \sim \mathcal{N}(0, I)$, $\hat{a} \sim \pi_{\text{edit}}(\cdot \mid s_{t+C}, a)$. In this way the candidate set is filtered once in noise space and once again after editing, while only a single VLA decode is required per backup regardless of N . Finally, Q_{ψ}^{dn} is kept consistent with the action critic by regressing it onto the value that $Q_{\phi'}$ assigns to the decoded chunk, with a stop-gradient on the target,

$$\begin{aligned}
 \mathcal{L}(Q_{\psi}^{\text{dn}}) &= \mathbb{E} \left[(Q_{\psi}^{\text{dn}}(s_{t+C}, \epsilon^*) \right. \\
 &\quad \left. - \text{sg}[Q_{\phi'}(s_{t+C}, \tilde{a}_{t+C:t+2C}^*)])^2 \right]
 \end{aligned} \tag{11}$$

so that noise-space selection inherits the ranking of the action-space critic as the latter improves. This allows a large number of action candidates to be used during training, which is helpful for accelerating training.

C. Implementation Details

Vision Backbone. At each timestep t , the state s_t comprises visual observations from the side and wrist cameras and robot joint positions, and is provided to both actor and critic. VLA actor processes the visual observations using its pretrained visual encoder. Following EXPO-FT [35], we equip critic with a separate, lightweight ResNet-50 encoder, which achieves high task performance at low computational cost. Full architectural details are provided in Section VII-E.

Training Procedure. For each task, we begin by configuring the camera setup and defining the reward signal, which can take the form of a rule-based criterion or a learned binary classifier. Before online RL begins, we collect a set of demonstrations and fine-tune the VLA using imitation

learning with Equation (7) under a randomized delay d , until it reaches a success rate of around 30% or higher. This data is then used to initialize the replay buffer. Starting from the supervised fine-tuned VLA policy, we then begin online RL training. The actor executes rollouts in the environment without human intervention, and the policy is updated after each episode, depending on the task requirements and available computational resources.

Reward Classifier. Reliable deployment requires an accurate and robust reward signal. To minimize task-specific reward engineering, we use a sparse binary reward for all tasks. Specifically, we define a rule-based classifier for each task that assigns a positive reward only when the task is successfully completed, and zero otherwise. The reward classifiers for all tasks are detailed in Section VII-D. This simple formulation avoids dense reward design while remaining effective across a diverse set of tasks.

V. EXPERIMENTS

We evaluate Real-Time EXPO-FT on ten environments in the Kinetix environment [26] and four dynamic real-world tasks, comparing against strong prior methods.

A. Baselines

RLPD [9]. RLPD is a sample-efficient off-policy reinforcement learning algorithm based on Soft Actor-Critic (SAC) [3], using balanced sampling between offline demonstrations and online experience. It has demonstrated strong performance across robotic manipulation tasks. RLPD uses a lightweight Gaussian policy as the actor, enabling efficient high-frequency control. Since it is naturally suited for real-time control, we run it as is without modification.

DSRL [31], DSRL w/ RTC. DSRL is a recent reinforcement learning method for finetuning pretrained diffusion and flow-matching policies. Rather than directly optimizing the policy weights, it learns to predict noise input to the pretrained policy. However, applying DSRL directly to high-frequency

control, such as 30 Hz, causes the policy to pause between action chunks, disrupting real-time execution. We therefore also evaluate DSRL with training-time RTC [16].

EXPO-FT [35], EXPO-FT w/ RTC. EXPO-FT is a state-of-the-art reinforcement learning framework for finetuning pretrained VLA policies. Similar to Real-Time EXPO-FT, it adopts EXPO [38] for policy improvement. However, directly applying EXPO-FT to high-frequency control can cause pauses between action chunks at 30 Hz. We therefore also evaluate EXPO-FT with training-time RTC [16] for a stronger comparison under real-time execution.

RLPD and DSRL are trained asynchronously, with the learner updating asynchronously at high update-to-data (UTD) ratios; this affords them substantially more gradient updates than EXPO-FT and Real-Time EXPO-FT, which perform updates per episode. This asymmetry favors RLPD and DSRL. We nonetheless retain this advantage for these prior methods throughout our experiments, as without it their performance degrades considerably, noting that even so, EXPO-FT and Real-Time EXPO-FT achieve higher performance despite the compute disadvantage.

B. Simulation Experiment

Task Setup. For our simulation experiments, we evaluate each approach on 10 dynamic tasks from the Kinetix benchmark [26], as shown in Figure 3, following the RTC setup [15]. The environments use force-based control with Gaussian action noise and feature dynamic motions such as catching and balancing, making dynamic control crucial for successful execution. Following the RTC setup [15], we pretrain the RTC flow-matching policy offline on 1M transitions, then finetune it online for 100k environment steps across all tasks. For all RL methods that use a base flow-matching policy, each call to the base policy incurs a 4-step inference delay during online finetuning and evaluation. The policy in RLPD and the edit policy in our method incurs no additional delay due to their lightweight nature, and is evaluated with zero inference delay. As a reference, we additionally report the pretrained BC policy under zero delay, which provides a reference on the performance achievable without inference latency.

Experiment Results. Now we present the simulation results. Figure 3 shows the training curves, while the full evaluation results are reported in Section VII-B. Real-Time EXPO-FT achieves an average success rate of 96.2% under the 4-step delay, substantially outperforming all delayed RL baselines. In particular, it improves over DSRL, DSRL w/ RTC, EXPO-FT, and EXPO-FT w/ RTC by 34.5, 20.1, 21.3, and 14.5 percentage points, respectively, demonstrating that explicitly addressing stale observations and delayed action generation is broadly effective across dynamic tasks and delayed settings. Importantly, Real-Time EXPO-FT not only compensates for inference delay but also surpasses the no-delay RLPD baseline on average. While the RLPD policy achieves an average success rate of 81.4% when evaluated with zero delay, Real-Time EXPO-FT reaches 96.2% while operating with a 4-step inference delay for the base flow-

matching policy and no delay for only the action edit. This result is particularly notable because the RLPD policy has access to the current observation when producing each action, whereas Real-Time EXPO-FT must generate the candidate actions based on delayed observations. We present additional experiments on the effectiveness of noise filtering evaluated in simulation in Section VII-A.

C. Real-World Experiment

Task Setup. In all real-world experiments, the robot is controlled in end-effector space using Cartesian and gripper velocity commands at 30 Hz. At each timestep, the policy receives two 224×224 RGB images from side- and wrist-mounted cameras, along with proprioceptive observations comprising the end-effector position and orientation. Environment resets are performed either automatically or by a human operator, depending on the task. We evaluate Real-Time EXPO-FT on four real-world tasks, shown in Figure 4. *Ball Balancing.* The Ball Balancing task requires the robot to control a black plate with a ping-pong ball on top. The robot must continuously rotate the plate to keep the ball near its center for several consecutive frames. The task is highly dynamic and stochastic, requiring smooth and reactive control to small changes in the ball’s position.

Dynamic Picking. The Dynamic Picking task requires the robot to pick up a block placed on a rotating plate. The block can start at arbitrary positions, resulting in different motion speeds and trajectories. The policy must infer the block’s motion and quickly reach and grasp it before it moves away.

Object Passing. The Object Passing task requires the robot to receive an object from another robot arm with random motion. The policy must continuously react to the motion of the other arm and rapidly move the end effector toward the object’s predicted position. The unpredictability of the object’s motion makes timely and reactive control essential. *Soccer Kicking.* The Soccer Kicking task requires the robot to kick a ball into a small goal while avoiding a moving defender. Successful execution requires precise spatial control and timing. The policy must determine both where to kick the ball and when to initiate the kick.

The VLA policy has an inference latency of approximately 67 ms on our server. For Ball Balancing, Object Passing, and Soccer Kicking, we introduce an additional 100 ms delay to simulate the higher inference latency associated with more constrained computing resources or a larger model, yielding a total latency of approximately 167 ms. For these three tasks, we set the delay d to 5, corresponding to approximately 167 ms. For Dynamic Picking, we retain the original inference latency of 67 ms and set the delay d to 3. Adding further latency reduces the success rates of all non-asynchronous baselines to nearly zero, as delayed gripper closure prevents timely grasping of the moving block.

Given the number of baselines and the computational cost, we cap training at 10 minutes of online robot interaction per task for all methods. We evaluate each task over 30 trials. Success is independently verified by a human observer. Further details on reward definitions, success detection,

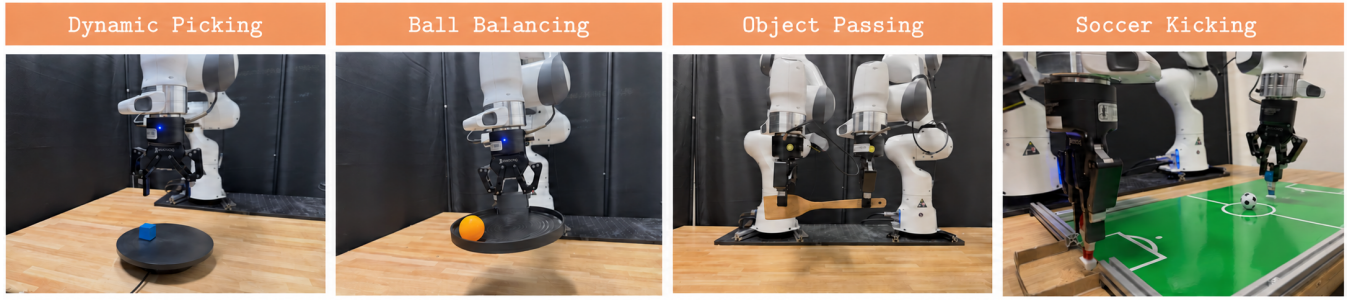


Fig. 4: **Four real-world manipulation tasks in our evaluation suite:** Dynamic Picking, Ball Balancing, Object Passing, and Soccer Kicking. All tasks require fast and reactive policies for dynamic environment changes.

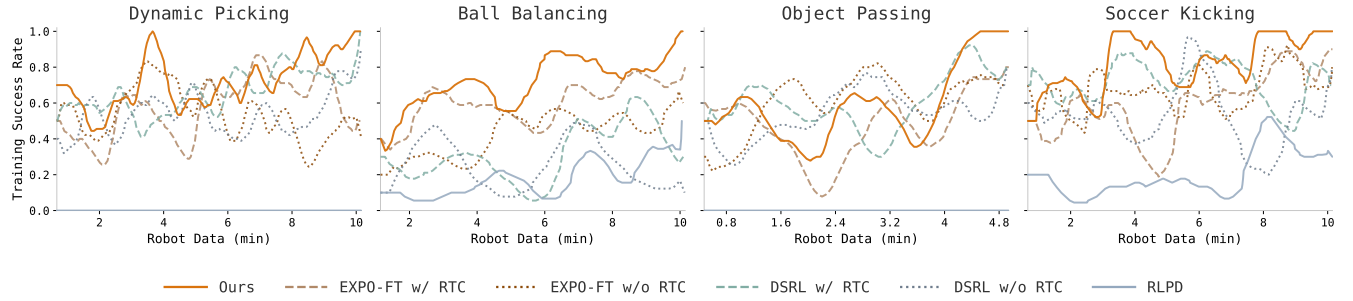


Fig. 5: **Training success across four real-world dynamic tasks.** We train each policy with at most 10 minutes of online robot data, or until one of the policies achieves 30/30 success during evaluation.

TABLE I: **Success rates on four real-world tasks.** Each task is evaluated over 30 trials. We train each policy until one policy reaches a 30/30 success rate or 10 minutes of online data have been collected and used for training.

Task	Success Rate (x/30). Training is capped at 10 minutes of online data.							
	SFT	SFT w/ RTC	RLPD	DSRL	DSRL w/ RTC	EXPO-FT	EXPO-FT w/ RTC	Real-Time EXPO-FT
Dynamic Picking	19/30	22/30	0/30	23/30	25/30	21/30	24/30	30/30
Ball Balancing	8/30	12/30	12/30	11/30	15/30	18/30	23/30	28/30 (10 min reached)
Object Passing	10/30	22/30	0/30	23/30	23/30	19/30	27/30	30/30
Soccer Kicking	13/30	16/30	6/30	16/30	17/30	17/30	26/30	28/30 (10 min reached)
Average	12.5/30	18/30	4.5/30	18.3/30	20/30	18.8/30	25/30	29/30

reset procedures, and task randomization are provided in Section VII-D.

Experiment Results. We now present our experimental results on the four dynamic real-world tasks. As shown in Figure 5 and Table I, Real-Time EXPO-FT consistently achieves near-perfect performance across all four tasks, with an average success rate of 29/30, substantially outperforming all prior methods. In particular, Ball Balancing exhibits substantial environmental randomness, where small perturbations in the ball’s motion can lead to significantly different future states, making rapid adaptation to the latest observation particularly important. On this task, Real-Time EXPO-FT achieves 28/30 success, while no prior method exceeds 23/30. Overall, Real-Time EXPO-FT significantly improves over prior methods.

D. Performance under varying delays and environment speeds

To further investigate the effectiveness of Real-Time EXPO-FT in highly dynamic, real-time settings, we conduct two

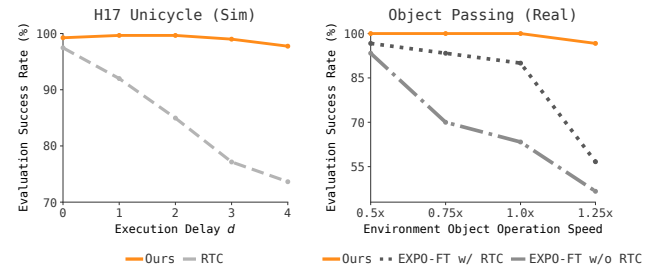


Fig. 6: **Success rates under varying delays and environment speeds.** We evaluate Real-Time EXPO-FT and the baselines on the H17 Unicycle task with varying delays d and on the Object Passing task with varying passing speeds.

additional experiments varying inference delays in the H17 Unicycle simulation task and object-passing speeds in the real-world Object Passing task. For varying delays, we train Real-Time EXPO-FT with different delays to simulate

variations in hardware capabilities and base-model inference speeds. As shown in Figure 6, Real-Time EXPO-FT maintains stable performance as the delay increases, whereas RTC’s performance deteriorates under longer delays. These results highlight the importance of accounting for inference latency in real-time control and demonstrate that our approach can better handle delays. For the object speed experiment, we evaluate Real-Time EXPO-FT against two EXPO-FT variants across different environment speeds. As shown in Figure 6, Real-Time EXPO-FT maintains a near-100% success rate across all tested speeds, whereas EXPO-FT without real-time degrade in performance as the passing speed increases. These results further demonstrate the importance of fast, reactive action edits for dynamic environments.

VI. DISCUSSION

We presented Real-Time EXPO-FT, a framework for RL finetuning of real-time VLA policies. Across a suite of challenging dynamic robotic tasks, Real-Time EXPO-FT demonstrates rapid sample-efficient adaptation to complex real-world dynamics. Despite these results, Real-Time EXPO-FT has limitations. First, a human provides environment resets in our experiments, which introduces operational burden; automating the reset process is an important direction for future work. Second, we use task-specific success detector following prior work; however, this requires designing classifier per task. While we do not explore alternative reward specifications in this work, identifying which reward formulation performs best remains an open question for future work.

VII. ACKNOWLEDGMENTS

This work was in part supported by NSF CAREER, NSF #1941722, RAI Institute, ONR grant N00014-22-1-2293, and ONR grant N00014-22-1-2621.

REFERENCES

- [1] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. *End-to-End Training of Deep Visuomotor Policies*. 2016. arXiv: 1504.00702 [cs.LG].
- [2] Shixiang Gu, Ethan Holly, Timothy Lillicrap, and Sergey Levine. “Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates”. In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. 2017, pp. 3389–3396. DOI: 10.1109/ICRA.2017.7989385.
- [3] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. 2018. arXiv: 1801.01290 [cs.LG].
- [4] Henry Zhu, Abhishek Gupta, Aravind Rajeswaran, Sergey Levine, and Vikash Kumar. *Dexterous Manipulation with Deep Reinforcement Learning: Efficient, General, and Low-Cost*. 2018. arXiv: 1810.06045 [cs.AI].
- [5] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. *Soft Actor-Critic Algorithms and Applications*. 2019. arXiv: 1812.05905 [cs.LG].
- [6] Ajay Mandlekar, Fabio Ramos, Byron Boots, Silvio Savarese, Li Fei-Fei, Animesh Garg, and Dieter Fox. *IRIS: Implicit Reinforcement without Interaction at Scale for Learning Control from Offline Robot Manipulation Data*. 2020. arXiv: 1911.05321 [cs.RO].
- [7] Xinyue Chen, Che Wang, Zijian Zhou, and Keith W. Ross. “Randomized Ensembled Double Q-Learning: Learning Fast Without a Model”. In: *International Conference on Learning Representations*. 2021.
- [8] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations*. 2022.
- [9] Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. “Efficient online reinforcement learning with offline data”. In: *Proceedings of the 40th International Conference on Machine Learning*. ICML’23. Honolulu, Hawaii, USA: JMLR.org, 2023.
- [10] Archit Sharma, Ahmed M. Ahmed, Rehaan Ahmad, and Chelsea Finn. *Self-Improving Robots: End-to-End Autonomous Visuomotor Reinforcement Learning*. 2023. arXiv: 2303.01488 [cs.RO].
- [11] Max Sobol Mark, Tian Gao, Georgia Gabriela Sampaio, Mohan Kumar Srirama, Archit Sharma, Chelsea Finn, and Aviral Kumar. *Policy Agnostic RL: Offline RL and Online RL Fine-Tuning of Any Class and Backbone*. 2024. arXiv: 2412.06685 [cs.LG].
- [12] Michal Nauman, Mateusz Ostaszewski, Krzysztof Jankowski, Piotr Miłoś, and Marek Cygan. *Bigger, Regularized, Optimistic: scaling for compute and sample-efficient continuous control*. 2024. arXiv: 2405.16158 [cs.LG].
- [13] Michael Psenka, Alejandro Escontrela, Pieter Abbeel, and Yi Ma. *Learning a Diffusion Model Policy from Rewards via Q-Score Matching*. 2024.
- [14] Lars Ankile, Zhenyu Jiang, Rocky Duan, Guanya Shi, Pieter Abbeel, and Anusha Nagabandi. *Residual Off-Policy RL for Finetuning Behavior Cloning Policies*. 2025. arXiv: 2509.19301 [cs.RO].
- [15] Kevin Black, Manuel Y Galliker, and Sergey Levine. “Real-Time Execution of Action Chunking Flow Policies”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025.
- [16] Kevin Black, Allen Z. Ren, Michael Equi, and Sergey Levine. *Training-Time Action Conditioning for Efficient Real-Time Chunking*. 2025. arXiv: 2512.05964 [cs.RO].
- [17] Yuhui Chen, Shuai Tian, Shugao Liu, Yingting Zhou, Haoran Li, and Dongbin Zhao. *ConRFT: A Reinforced Fine-tuning Method for VLA Models via Consistency Policy*. 2025. arXiv: 2502.05450 [cs.RO].

- [18] Perry Dong, Alec M. Lessing, Annie S. Chen, and Chelsea Finn. *Reinforcement Learning via Implicit Imitation Guidance*. 2025. arXiv: 2506.07505 [cs.LG].
- [19] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, Vishnu Mano, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alex Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haohuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, and Zhiyuan Zhou. $\pi_{0.6}^*$: *a VLA That Learns From Experience*. 2025. arXiv: 2511.14759 [cs.LG].
- [20] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: *a Vision-Language-Action Model with Open-World Generalization*. 2025. arXiv: 2504.16054 [cs.LG].
- [21] Alexander Khazatsky et al. *DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset*. 2025. arXiv: 2403.12945 [cs.RO].
- [22] Guanxing Lu, Wenkai Guo, Chubin Zhang, Yuheng Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. *VLA-RL: Towards Masterful and General Robotic Manipulation with Scalable Reinforcement Learning*. 2025. arXiv: 2505.18719 [cs.RO].
- [23] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. *SERL: A Software Suite for Sample-Efficient Robotic Reinforcement Learning*. 2025. arXiv: 2401.16013 [cs.RO].
- [24] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. *Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning*. 2025. arXiv: 2410.21845 [cs.RO].
- [25] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. *Running VLAs at Real-time Speed*. 2025. arXiv: 2510.26742 [cs.RO].
- [26] Michael Matthews, Michael Beukman, Chris Lu, and Jakob Nicolaus Foerster. “Kinetix: Investigating the Training of General Agents through Open-Ended Physics-Based Control Tasks”. In: *The Thirteenth International Conference on Learning Representations*. 2025.
- [27] NVIDIA, : Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llonet, Loic Magne, Ajay Mandelkar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzheng Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. *GR00T N1: An Open Foundation Model for Generalist Humanoid Robots*. 2025. arXiv: 2503.14734 [cs.RO].
- [28] Allen Z. Ren, Justin Lidard, Lars Lien Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. “Diffusion Policy Policy Optimization”. In: *The Thirteenth International Conference on Learning Representations*. 2025.
- [29] Kohei Sendai, Maxime Alvarez, Tatsuya Matsushima, Yutaka Matsuo, and Yusuke Iwasawa. *Leave No Observation Behind: Real-time Correction for VLA Action Chunks*. 2025. arXiv: 2509.23224 [cs.RO].
- [30] Gemini Robotics Team et al. *Gemini Robotics 1.5: Pushing the Frontier of Generalist Robots with Advanced Embodied Reasoning, Thinking, and Motion Transfer*. 2025. arXiv: 2510.03342 [cs.RO].
- [31] Andrew Wagenmaker, Mitsuhiro Nakamoto, Yunchu Zhang, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. “Steering Your Diffusion Policy with Latent Space Reinforcement Learning”. In: *Conference on Robot Learning* (2025).
- [32] Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. “EfficientVLA: Training-Free Acceleration and Compression for Vision-Language-Action Models”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025.
- [33] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : *A Vision-Language-Action Flow Model for*

- General Robot Control*. 2026. arXiv: 2410.24164 [cs.LG].
- [34] Kang Chen, Zhihao Liu, Tonghe Zhang, Zhen Guo, Si Xu, Hao Lin, Hongzhi Zang, Xiang Li, Quanlu Zhang, Zhaoifei Yu, Guoliang Fan, Tiejun Huang, Yu Wang, and Chao Yu. π_{RL} : *Online RL Fine-tuning for Flow-based Vision-Language-Action Models*. 2026. arXiv: 2510.25889 [cs.LG].
- [35] Perry Dong, Kuo-Han Hung, Tian Gao, Dorsa Sadigh, and Chelsea Finn. *EXPO-FT: Sample-Efficient Reinforcement Learning Finetuning for Vision-Language-Action Models*. 2026. arXiv: 2605.25477 [cs.RO].
- [36] Perry Dong, Kuo-Han Hung, Alexander Szwedlow, Dorsa Sadigh, and Chelsea Finn. *TQL: Scaling Q-Functions with Transformers by Preventing Attention Collapse*. 2026. arXiv: 2602.01439 [cs.LG].
- [37] Perry Dong, Yueru Jia, Chelsea Finn, and Dorsa Sadigh. *Q-Learning With World Models*. 2026. arXiv: 2608.17163 [cs.LG].
- [38] Perry Dong, Qiyang Li, Dorsa Sadigh, and Chelsea Finn. “EXPO: Stable Reinforcement Learning with Expressive Policies”. In: *The Fourteenth International Conference on Learning Representations*. 2026.
- [39] Perry Dong, Ron Polonsky, Dorsa Sadigh, and Chelsea Finn. *Do You Really Need to Pretrain Q-Functions for Online RL Fine-Tuning?* 2026. arXiv: 2607.27203 [cs.LG].
- [40] Perry Dong, Alexander Szwedlow, Dorsa Sadigh, and Chelsea Finn. *FASTER: Value-Guided Sampling for Fast RL*. 2026. arXiv: 2604.19730 [cs.LG].
- [41] Perry Dong, Chongyi Zheng, Chelsea Finn, Dorsa Sadigh, and Benjamin Eysenbach. *Value Flows*. 2026. arXiv: 2510.07650 [cs.LG].
- [42] Hai Jiang, Yixian Zou, Binbin Liang, Boqian Liu, Fanman Meng, and Shuaicheng Liu. *FutureRTC: Real-Time Robot Execution with Anticipatory-Conditioned Action Chunking*. 2026. arXiv: 2607.24008 [cs.RO].
- [43] Kun Lei, Huanyu Li, Dongjie Yu, Zhenyu Wei, Lingxiao Guo, Zhennan Jiang, Ziyu Wang, Shiyu Liang, and Huazhe Xu. *RL-100: Performant Robotic Manipulation with Real-World Reinforcement Learning*. 2026. arXiv: 2510.14830 [cs.RO].
- [44] Qiyang Li and Sergey Levine. “Q-Learning with Adjoint Matching”. In: *The Fourteenth International Conference on Learning Representations*. 2026.
- [45] Yuxiang Lu, Zhe Liu, Xianzhe Fan, Zhenya Yang, Jinghua Hou, Junyi Li, Kaixin Ding, and Hengshuang Zhao. “FASTER: Rethinking Real-Time Flow VLAs”. In: *arXiv preprint arXiv:2603.19199* (2026).
- [46] Jiahui Niu, Kefan Gu, Yucheng Zhao, Shengwen Liang, Tiancai Wang, Xing Hu, Ying Wang, and Huawei Li. *Realtime-VLA FLASH: Speculative Inference Framework for Diffusion-based VLAs*. 2026. arXiv: 2605.13778 [cs.RO].
- [47] Sungjae Park and Shubham Tulsiani. πR^2 : *Reactive Real-time Flow Policies*. 2026. arXiv: 2607.26055 [cs.RO].
- [48] Sarvesh Patil, Mitsuhiko Nakamoto, Manan Agarwal, Shashwat Saxena, Jesse Zhang, Giri Anantharaman, Cleah Winston, Chaoyi Pan, Douglas Chen, Nai-Chieh Huang, Zeynep Temel, Oliver Kroemer, Sergey Levine, Abhishek Gupta, Hongkai Dai, Paarth Shah, and Max Simchowitz. *OGPO: Sample Efficient Full-Finetuning of Generative Control Policies*. 2026. arXiv: 2605.03065 [cs.LG].
- [49] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. *VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference*. 2026. arXiv: 2512.01031 [cs.RO].
- [50] Wenli Xiao, Haotian Lin, Andy Peng, Haoru Xue, Tairan He, Zhengyi Luo, Yuqi Xie, Fengyuan Hu, Linxi Fan, Guanya Shi, and Yuke Zhu. “Self-Improving Vision-Language-Action Models with Data Generation via Residual RL”. In: *The Fourteenth International Conference on Learning Representations*. 2026.
- [51] Charles Xu, Jost Tobias Springenberg, Michael Equi, Ali Amin, Adnan Esmail, Sergey Levine, and Liyiming Ke. *RL Token: Bootstrapping Online RL with Vision-Language-Action Models*. 2026. arXiv: 2604.23073 [cs.LG].
- [52] Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. *ReinFlow: Fine-tuning Flow Matching Policy with Online Reinforcement Learning*. 2026. arXiv: 2505.22094 [cs.RO].

A. Noise-level Filtering Studies

To handle the high randomness of dynamic tasks, we empirically find that a larger sampling number such as $N = 32$ is often required. However, setting a large N such as 32 introduces a substantial computational burden during training. To address this issue, we incorporate the noise-level filtering technique introduced in Equation (10) to filter action candidates during Bellman updates.

To evaluate the effectiveness and efficiency of noise-level filtering, we compare using and not using noise-level filtering and instead filters over the fully denoised actions. As shown in the Figure 7, using noise filtering learns significantly more efficiently than not using it under the same training compute, demonstrating that noise-level filtering can effectively improve training by enabling a large N without incurring the computational cost.

B. Full Simulation Experiment Results

Here, we provide detailed simulation experiment results, including evaluations of all baselines. For each method, we conduct 100 trials in each environment and average the results across four random seeds. As shown in Table II, Real-Time EXPO-FT outperforms all baselines across the evaluated environments.

C. Detailed Simulation Task Settings

1) *Environment and Base Policy*: For simulation, we use the vector-state (symbolic) Kinetix benchmark, where no VLA is involved and the base policy is a pretrained state-based flow-matching policy. We evaluate on 10 environments: `car_launch`, `cartpole_thrust`, `catapult`, `catcher_v3`, `h17_unicycle`, `hard_lunar_lander`, `mjc_half_cheetah`, `trampoline`, `chain_lander`, and `grasp_easy`. We use four random seeds and a budget of 100k environment steps per run. The environment applies Gaussian action noise with a standard deviation of 0.1, matching the reference data generation and evaluation rollouts. Success is determined by the environment’s episode-solved flag.

The base policy is the publicly released per-level behavior-cloned flow policy from the real-time-chunking [15] Kinetix benchmark [26]. It uses a channel dimension of 256, a channel hidden dimension of 512, a token hidden dimension of 64, four layers, an action-chunk length of $H = 8$, and five flow-matching steps during training. The policy is not delay-conditioned. At rollout and during the critic backup, we sample the policy using 10 Euler denoising steps.

Online fine-tuning of the base policy uses AdamW with a learning rate of 3×10^{-4} , weight decay of 10^{-2} , gradient-norm clipping of 10, and a 1000-step warmup. We use a prefix-conditioned flow-matching objective with $d \sim \text{Unif}\{0, \dots, 4\}$, where the prefix length is resampled independently for each training example. Unlike the real-world setting, where the online prefix length is fixed to the deployment delay, the simulation setting resamples the prefix length during training.

2) *Learner Configuration*: The critic, filter, and edit policy use the same overall architecture as in the real-world setting, including a REDQ ensemble of 10 networks with two networks subsampled for each target estimate, LayerNorm, and hidden dimensions (256, 256, 256). For simulation, the visual encoder is replaced by an MLP state encoder with a 256-dimensional output. Different from the real-world settings, the base-policy update is not restricted to successful episodes, the BC loss Equation (7) is applied to all the rollout and demo data.

3) State-Based Baselines:

a) *DSRL (state)*: DSRL applies SAC in the flow policy’s noise space over the frozen per-level base policy, with $M = 1.5$, target entropy 0, no entropy term in the Bellman backup, and no demonstration data. We evaluate three settings: no inference delay; delay $d = 4$ with real-time chunking, where the in-flight prefix is inpainted into chunk positions $[0 : 4]$ and the window $[4 : 8]$ is executed; and delay $d = 4$ with naive replanning, which uses the same stale observation without prefix conditioning. The third setting isolates the effect of prefix conditioning from the effect of acting on stale observations.

b) *RLPD (state)*: RLPD uses SAC and learns directly from scratch, with 50% demonstration data in every batch and zero inference delay. The pretrained flow checkpoint is used only for observation preprocessing and reference evaluation and does not contribute to the learned policy. Critic hyperparameters are identical to those of Real-Time EXPO-FT, so the two methods differ only in their actor parameterization and use of the pretrained policy.

D. Detailed Real-World Task Settings

1) *Task Setting Description*: Here, we provide detailed task settings for the four real-world tasks evaluated in our experiments, including the task objectives, success detector implementation and definition, reward function, initial-state randomization, camera configuration, and demonstration collection procedure.

All four real-world tasks use the same single-arm DROID [21] setup with a 30 Hz control rate and two policy camera views, consisting of one exterior camera and one wrist-mounted camera. Each image is resized to 224×224 . Rewards are sparse and binary: an automatic detector emits $r = 1$ on the step at which it declares success and terminates the episode. A timeout terminates the episode with $r = 0$ and is treated as a failure. Episodes are additionally capped at a task-specific horizon.

2) *Success Detectors*: All success detectors operate on the robot’s own observation stream and therefore require no external instrumentation. Dynamic Picking is detected proprioceptively: a successful lift is declared when the end-effector height exceeds 0.30 while the gripper is closed beyond a mid-aperture threshold for 5 consecutive steps. Soccer Kicking and Ball Balancing use analogous wrist-view-based detectors, with Ball Balancing additionally requiring the ball to remain within a specified center tolerance for 10 consecutive frames. Object Passing uses a wrist-view-based

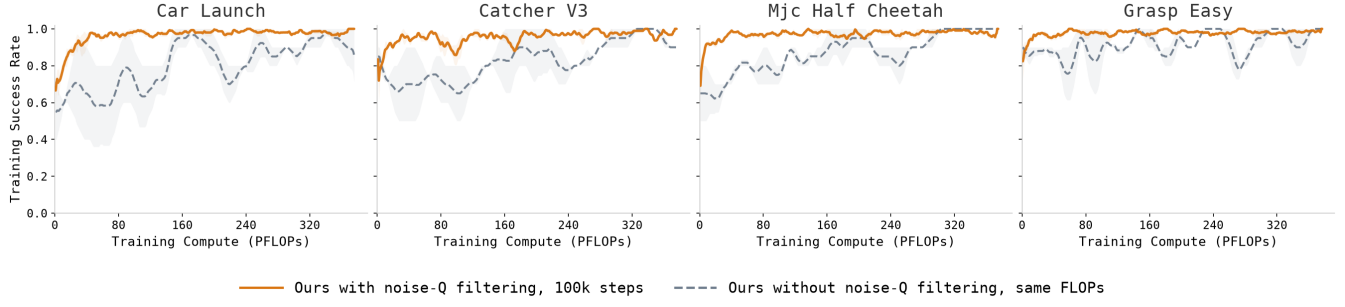


Fig. 7: **Comparison of noise-level filtering.** We compare Real-Time EXPO-FT with and without noise-level filtering, where the latter uses only Q-level filtering during the Bellman backup. The x-axis shows training compute, allowing us to compare the efficiency of the two approaches.

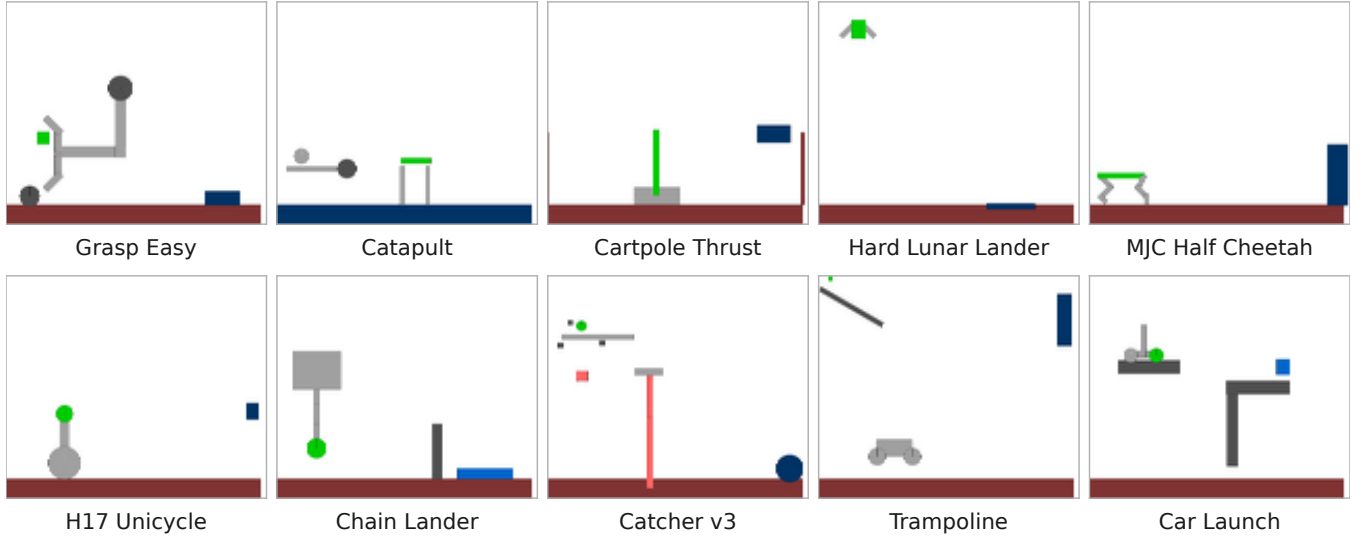


Fig. 8: **10 Kinetix simulation tasks [26] evaluated in our experiments.**

TABLE II: Full simulation results on the Kinetix benchmark: success rate (%). RL results average four random seeds $\times$ 100 evaluation episodes; BC deploys the pretrained policy without fine-tuning, and RTC adds real-time chunking on top of it (512 episodes each). Delay-4 methods replan every 4 steps under a 4-step inference delay; RLPD supports only zero delay. **Bold** marks every RL method within $0.95\times$ the best RL result on that task (BC and RTC are excluded from the comparison).

Task	Delay = 0		Delay = 4						
	BC	RLPD	BC	RTC	DSRL	DSRL w/ RTC	EXPO-FT	EXPO-FT w/ RTC	Real-Time EXPO-FT
Car Launch	94%	97%	51%	70%	76%	79%	44%	64%	98%
Cartpole Thrust	100%	57%	37%	92%	49%	85%	99%	96%	98%
Catapult	62%	83%	31%	42%	36%	49%	18%	47%	93%
Catcher	97%	90%	23%	90%	40%	71%	90%	85%	97%
Unicycle	97%	90%	58%	74%	48%	86%	48%	87%	99%
Hard Lunar Lander	95%	70%	57%	87%	75%	79%	95%	90%	94%
Half-Cheetah	88%	93%	72%	75%	86%	85%	75%	79%	98%
Trampoline	84%	47%	68%	85%	43%	42%	93%	83%	94%
Chain Lander	95%	90%	94%	94%	92%	96%	90%	94%	92%
Grasp	95%	97%	61%	91%	72%	89%	97%	92%	99%
Average	90.7%	81.4%	55.2%	80.0%	61.7%	76.1%	74.9%	81.7%	96.2%

detector to determine whether the robot has successfully grasped the object: a success is declared when the gripper is closed and the object remains detected for 5 consecutive

control steps.

3) *Additional Critic Inputs:* For two tasks, a small number of quantities already measured by the success detector are

written into unused slots of the proprioceptive state vector. These quantities are available to the critic, the noise- Q filter, and the edit policy. The base VLA’s own state input remains unchanged, so the supervised checkpoint and normalization statistics are unaffected.

Ball Balancing exposes the plate center, ball position, and ball velocity. Soccer Kicking exposes the keeper’s position and velocity.

For Ball Balancing, we additionally remove the vertical (z) proprioceptive dimension from the critic input because it drifts monotonically with episode time and may allow the value function to exploit episode-time information. Both DSRL and RLPD receive the same privileged state dimensions in their critics.

4) *Observation Layout*: Three tasks use one exterior view and one wrist view. The critic encoder consumes these views as a six-channel tensor. Ball Balancing instead uses a three-frame stack of the exterior view at t , $t - k$, and $t - 2k$, while dropping the wrist view from the policy input. Consequently, the critic encoder consumes nine channels, and the VLA receives the corresponding three-image configuration.

5) *Task Full Execution Strips*: We visualize the full execution trajectories of the evaluated tasks. As shown in Figure 9, each strip illustrates the temporal progression of the task from initiation to completion, providing a qualitative view of the robot’s behavior throughout the entire execution.

6) *Task Initial-State Randomization Space*: We visualize the task initial-state randomization space to illustrate the range of initial object positions. As shown in Figure 10, the randomized space is highlighted by the orange boxes, capturing the randomization applied to both the objects and the robot.

E. Detailed Training Settings

1) *Base Policy Initialization*: We instantiate Real-Time EXPO-FT with $\pi_{0.5}$ [20] as the base policy. The model uses a LoRA [8] configuration with a `gemma_2b_lora` language backbone and a `gemma_300m_lora` action expert. The padded action dimension is 32, the action horizon is $H = 16$, and the output action dimension is 7. Proprioceptive state is provided to the VLA in Cartesian form, and input images are resized to 224×224 .

The initialization is a task-specific prefix-conditioned real-time-chunking LoRA supervised fine-tuning of $\pi_{0.5}$ using the corresponding task demonstrations and normalization statistics. During supervised training, the per-example prefix length is sampled as $d \sim \text{Unif}\{0, \dots, d_{\max}\}$. The first d chunk positions are provided with clean ground-truth actions at flow time $\tau = 1$, and the flow-matching loss is applied only to the remaining $H - d$ positions. This exposes the model to both the boot regime ($d = 0$) and the delayed inpainting regime during supervised training. The image encoder is trainable during this stage.

2) Model Structure and Learned Components:

a) *Value function*: The critic is a REDQ-style [7] ensemble of 10 Q -networks with LayerNorm and three hidden layers of width 256. Every Q evaluation, including both the

Bellman target and rollout-time action selection, samples two networks uniformly from the target ensemble and takes their minimum. The target ensemble follows the online ensemble using Polyak averaging with $\tau_Q = 5 \times 10^{-3}$. For Kick and Balance, we additionally train the critic on reward windows containing terminal transitions, whose targets consist only of the observed reward.

b) *Critic visual encoder*: The critic uses a pre-activation ResNetV2 with basic, non-bottleneck residual blocks, stage depths (3, 4, 6, 3), GroupNorm with four groups, and 64 base filters that double at each stage to 512. At 224×224 resolution, the stem consists of a stride-2 7×7 convolution followed by max pooling. A single encoder consumes the camera views as a channel-stacked tensor, using six channels for two views and nine channels for Balance’s three-frame stack. The resulting representation is projected to a 512-dimensional image embedding using a Dense+LayerNorm head. Proprioception is embedded into 64 dimensions and concatenated with the image embedding and flattened action chunk for Q -value prediction.

c) *Edit policy*: The edit policy is a tanh-squashed Gaussian over the flattened execution window, with dimension $D = C \times 7$. For $C = 8$, this gives $D = 56$. The policy is conditioned on the critic’s image embedding, proprioceptive embedding, and the base action chunk being corrected. It reuses the critic’s image encoder and contains three hidden layers of width 256. Its output lies in $[-1, 1]^D$ and is multiplied by a task-specific edit scale before being added to the base action chunk. For Dynamic Pick, the rotational components of the edit are masked to zero, so the edit acts only on translation and gripper dimensions.

d) *Action selection*: At each replan boundary, the base policy draws $N = 32$ stochastic action chunks using 10 Euler denoising steps. Because the VLM prefix is shared across noise samples, it is computed only once. Each base chunk receives one sampled edit, producing 64 candidates in total: 32 base candidates and 32 edited candidates. The executed chunk is selected deterministically using the $\arg \max$ of the minimum-over-two-subsampled target Q value. No softmax is applied over candidates.

e) *Noise- Q backup filter*: Denoising 32 candidates during every Bellman backup would substantially increase computational cost. We therefore pre-filter candidates in noise space. A filter critic $Q_f(s', \epsilon)$ scores 32 raw Gaussian seeds in the padded model action space ($H \times 32$). The highest-scoring seed is denoised using $\arg \max$ with a sampling temperature of zero, and one edit candidate is sampled from the resulting action chunk. The outer target- Q maximization then considers one base candidate and one edited candidate.

Q_f is a two-network ensemble trained at every critic step using MSE regression onto the outer target critic’s Q value for the denoised survivor, with the target stop-gradient applied. Because the regression target is supervised, Q_f does not require a target network. The filter additionally conditions on the delayed observation used by the base policy, consisting of the image embedding and proprioception. Rollout action selection is unaffected by this filter.

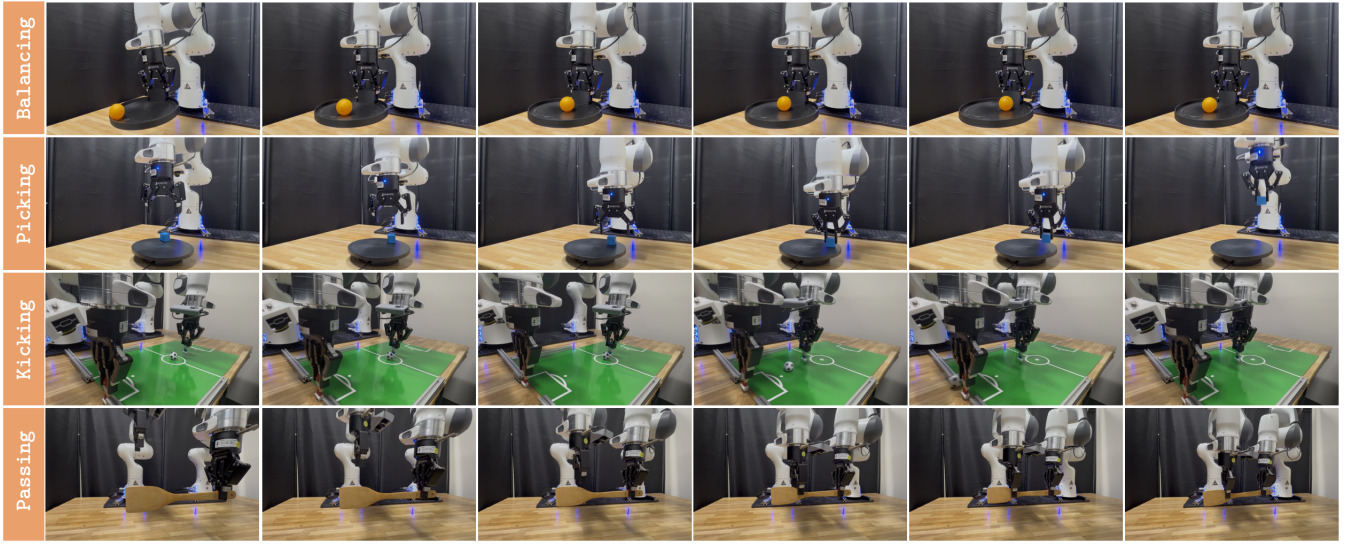


Fig. 9: Full execution strips for the evaluated tasks. Each strip shows the temporal progression of a complete task execution, from initialization to successful completion.

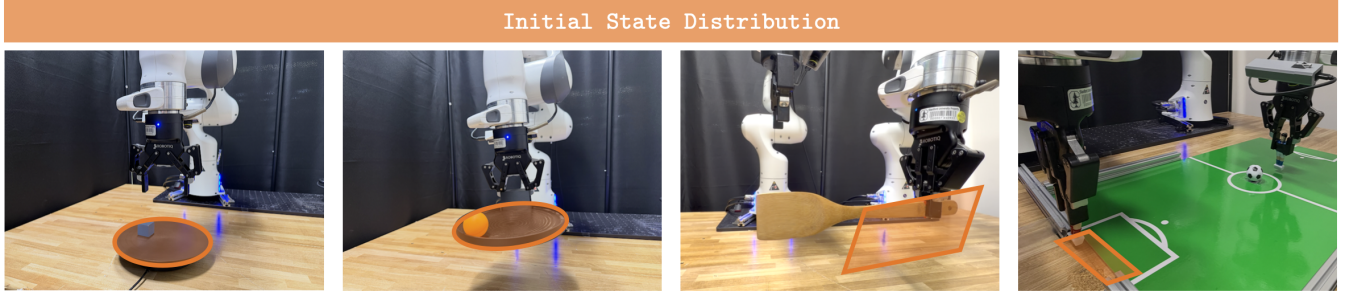


Fig. 10: Initial-state randomization spaces for the real-world tasks. The orange boxes indicate the regions within which object and robot initial states are randomized.

TABLE III: Shared optimization hyperparameters used across all real-world experiments.

Hyperparameter	Value
Optimizer (critic, filter, edit policy, temperature)	Adam, 3×10^{-4}
Optimizer (base VLA)	AdamW, 2.5×10^{-5} , clip 1.0
Critic target update τ_Q	5×10^{-3}
Base-policy Polyak copy τ_π	10^{-3}
Initial temperature α_0	0.01
Target entropy	$-D/2$, $D = C \times 7$
Critic minibatch size	64
Update-to-data ratio	20
Q -ensemble size / subsample	10/2
Filter-critic ensemble size	2
Action-chunk horizon H	16
Base candidates N / edit candidates	32/32
Backup noise seeds / survivors / edits	32/1/1
Denoising steps	10
Image / state embedding dimensions	512/64
Hidden layers	(256, 256, 256)

f) Base-policy fine-tuning: The base VLA is fine-tuned using prefix-conditioned flow-matching behavior cloning on successful episodes, including demonstrations and successful online episodes. Exactly one base-policy update is performed per update call. The prefix length is deterministic during online training: $d = 0$ for transitions in the first chunk of

an episode and d equal to the deployment delay thereafter. Unlike supervised initialization, the online prefix length is therefore not resampled.

The trainable parameters include the LoRA adapters in the language model as well as all parameters outside the frozen non-LoRA language-model weights, including the SigLIP

TABLE IV: Task-specific hyperparameters for Real-Time EXPO-FT. K denotes the number of collected transitions per update call. “Prior data” indicates whether demonstrations are sampled as a fixed fraction of each critic batch or seeded into the online replay buffer. Environment steps denote the total budget of the reported run.

Task	Edit scale	Replan C	Delay d	K	Prior data	Env. steps
Dynamic Picking	0.1	8	3	25	Seeded in buffer	$\sim 18k$
Soccer Kicking	0.05	8	5	20	50% of each batch	$\sim 18k$
Ball Balancing	0.1	8	5	30	Seeded in buffer	$\sim 18k$
Object Passing	0.1	8	5	30	Seeded in buffer	$\sim 5k$

vision tower and projection layers. The frozen language-model parameters are maintained in bfloat16. The VLA is optimized using the AdamW configuration of the underlying implementation [20], with $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay 10^{-10} , and gradient-norm clipping at 1.0. We use a constant learning rate of 2.5×10^{-5} and no EMA. A Polyak copy of the base-policy parameters is maintained with $\tau_\pi = 10^{-3}$ but is not used by the current Bellman backup, which samples next actions from the live base policy.

g) *Entropy and temperature*: The learnable temperature is initialized at $\alpha_0 = 0.01$ and optimized with Adam at 3×10^{-4} using a target entropy of $-D/2$, where $D = C \times 7$ is the edit-policy dimension. Entropy affects only the edit-policy objective and does not appear in the Bellman backup.

h) *Image augmentation*: Both current and next observations are augmented independently. For each view, we apply a 95% random crop followed by resizing to 224×224 , a random rotation in $[-5^\circ, 5^\circ]$, and color jitter with brightness, contrast, and saturation changes of ± 0.1 . The same augmentation procedure is applied to critic, filter, edit-policy, and base-policy inputs.

3) *Latency Model*: We study two ways of realizing inference latency at a 30 Hz control rate. In the *wall-clock* condition, we set $d = 0$ and add 100 ms of real sleep to every sample actions call for soccer kicking, ball balancing, and object passing, approximating the compute time of running the $\pi_{0.5}$ model on a typical edge GPU. For dynamic picking, we do not inject additional wall-clock latency, as the task is highly dynamic and even modest additional latency causes the baseline methods to fail almost entirely, making the comparison less informative. In the *chunk-delay* condition, the policy observes a d -step-old observation, the d actions currently in flight are inpainted into chunk positions $[0, d)$ as a clean prefix, and the window $[d, d + C)$ is executed. We use $d = 3$ for dynamic picking and $d = 5$ for the other tasks, corresponding to approximately 100 ms and 167 ms, respectively, and matching the inference-time settings used in the wall-clock condition.

4) *Optimization Hyperparameters*: Table III lists the hyperparameters shared across all real-world experiments, while Table IV lists the task-specific settings.

All learned components other than the base VLA, including the Q ensemble, filter critic, edit policy, and temperature, use Adam with a learning rate of 3×10^{-4} . The base VLA uses the AdamW configuration described above. Each update call samples batch size $\times$ UTD transitions and performs the specified number of critic gradient steps on disjoint

minibatches, followed by exactly one base-policy step, one edit-policy step, and one temperature step. Thus, the base-policy-to-critic gradient-step ratio is 1 : 20 rather than 1 : 1.

Update calls are accumulated at a rate of one per K collected transitions and flushed at episode boundaries. Training begins only after 10 episodes have been completed.

5) *Evaluation Protocol*: Each method is evaluated from its final checkpoint unless otherwise specified, under the same latency condition used during training.

F. Baseline Implementations

a) *RLPD* [9]: We follow the SERL [23] setup for RLPD [9], using the same $\pi_{0.5}$ -style observation pipeline as our real-robot stack. No VLA is used in the policy. The policy is a tanh-Gaussian distribution over a single 7-dimensional action and is queried at every control step. It runs with zero inference delay, so its Bellman backup uses γ per environment step rather than γ^C .

We use hidden dimensions (256, 256, 256), Adam with a learning rate of 3×10^{-4} , $\gamma = 0.99$ (0.997 for Kick), minibatch size 256, UTD ratio 4, initial temperature 0.1, target entropy $-7/2 = -3.5$, and no entropy term in the Bellman backup. The critic uses the same REDQ-style ensemble as Real-Time EXPO-FT, with 10 networks and two subsampled for each target estimate, together with LayerNorm, a 512-dimensional image latent, and a 64-dimensional state latent. We use the same augmentation procedure, but a smaller ResNetV2 visual backbone with stage depths (1, 1, 1, 1).

Demonstrations constitute 50% of every critic batch. Training runs in an asynchronous learner thread that is not rate-limited by the environment. We report the measured number of optimizer steps per environment step for each run rather than assuming a fixed ratio.

b) *DSRL* [31], *DSRL w/ RTC*: DSRL uses the frozen $\pi_{0.5}$ policy loaded from the same prefix-conditioned supervised checkpoint used to initialize Real-Time EXPO-FT. SAC operates in noise space: the actor outputs a single 32-dimensional noise vector corresponding to the padded model action dimension, squashed as $M \tanh(u)$ with $M = 1.0$ and tiled across the 16-step action horizon.

The critic is parameterized as $Q(\text{enc}(s), \epsilon)$ and uses the same ResNetV2 encoder as Real-Time EXPO-FT, with stage depths (3, 4, 6, 3), width 64, input resolution 224×224 , a 512-dimensional image latent, a 64-dimensional state latent, hidden dimensions (256, 256, 256), LayerNorm, and a REDQ ensemble of 10 networks with two subsampled for the target.

State information is included in the critic, and the same image augmentation is applied.

The actor learning rate is 10^{-4} , while the critic and temperature use 3×10^{-4} . We use $\gamma = 0.99$ (0.997 for Kick), $\tau_Q = 5 \times 10^{-3}$, minibatch size 64, UTD ratio 20, initial temperature 0.01, target entropy 0, and no entropy term in the Bellman backup.

DSRL executes $C = 8$ steps per replan. Under nonzero delay, it uses the same real-time-chunking prefix inpainting as Real-Time EXPO-FT, with SAC noise applied only to the postfix. The Bellman backup requires no modification because DSRL operates entirely in noise space. DSRL is trained purely online without demonstration data because demonstrations do not contain the corresponding noise labels. For DSRL w/ RTC, we use the same delay as Real-Time EXPO-FT and same optimization parameters as DSRL.

c) EXPO-FT [35], EXPO-FT w/ RTC: EXPO-FT uses the same learner architecture, network sizes, candidate counts, filter, optimizer settings, and prior-data configuration as Real-Time EXPO-FT. The only difference is that EXPO-FT is delay-unaware: it is trained and evaluated with $d = 0$ while incurring 100 ms of real inference latency. This setting isolates the contribution of delay-aware chunking from the underlying EXPO optimization. For EXPO-FT with RTC, we use the same delay and optimization parameters as Real-Time EXPO-FT.